15,471,951 members Sign in

CODE PROJECT For those who code

articles Q&A forums stuff lounge ? Search for articles, questions,

A Thread Pooling and Task Queuing Demonstration Using Message Passing



honey the codewitch

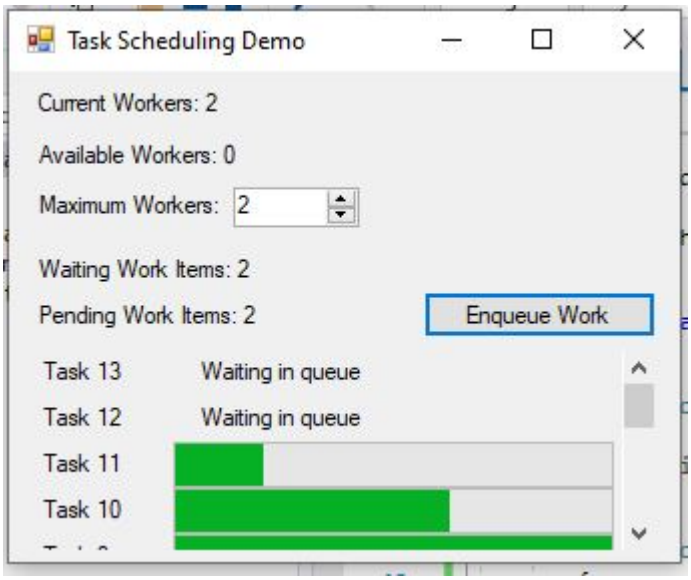
15 Jul 2020 MIT

Rate me: 4.47/5 (9 votes)

How to implement a complicated multithreaded message passing scenario

This article presents one scenario for queuing work among a pool of threads and reporting progress using message passing. It uses a raw approach rather than using ThreadPool and Task in order that we can explore the concepts.

Download source code - 18.5 KB



Introduction

With this article, I endeavor to show you how to implement a complex multithreaded scenario using message passing for synchronization and thread pooling for efficiently allocating threads. While we could potentially use `ThreadPool` and `Task` to accomplish much, if not all of this, it doesn't necessarily use message passing. Meanwhile, the concepts and techniques I illustrate are applicable to most programming languages, unlike the task framework and the build in thread pool. Mostly this is about concepts, not so much production code.

Conceptualizing this Mess

Message passing is a tried and true way to communicate in a safe manner between threads. Basically, it works by establishing a thread-safe queue used for enqueuing messages. Both the client dispatcher and the workers have access to this queue. The client dispatcher is primarily responsible for enqueing messages while the workers pull messages out of the queue.

If we want bidirectional communication, we can establish a second queue going the other direction. We do that in the demo in order to report things back to the client like the progress of work items.

Furthermore, in order to prevent the system scheduler from being bogged down or creating and destroying threads all the time, it's a good idea to limit the number of threads you use and pool/recycle the ones you already have. This project handles that as well.

Message Passing

First, a queue is simply a collection that adds items to the back and removes items from the front. When you remove an item, you have an opportunity to examine it. It's just a collection, basically. The one we use happens to be safe to call from across threads and this is critical. We use this queue to hold pending messages to be processed by the workers. There is one worker message queue that is shared between all workers.

Second, a semaphore is a synchronization tool that holds a count. The count is decremented each time a thread completes a `Wait()` on it and incremented each time a thread calls `Release()` on it. The `Wait()` call blocks unless the count is non-zero. We use this to signal when one or more messages are waiting in the queue. The count of the semaphore is the same as the number of messages in the queue. This way, our worker threads can sleep by waiting (with `Wait()`) until a message arrives. Like with the message queue, all the workers share the same semaphore.

Any time we need something done and that task is being requested from a different thread, we need to send a message. We have workers and a client. The client's primary job is to dispatch to workers. In order to pass a message to a running worker, the client must `Enqueue()` a `WorkerMessage` and call `Release(1)` on the semaphore to signal a waiting message which increments the semaphore's count. Each of the idle worker threads is waiting for the semaphore count to go non-zero. Once it does, the next thread waiting on the semaphore wakes up. It receives the message and removes it from the queue in order to process it.

Complicating things a little, workers can also send messages back to the client. This is so we can get various notifications like when a worker message is being processed or when it's complete, or progress on the worker message. Therefore, the client must also process its own messages, just like a worker does.

Finally, each message has an `Id` that uniquely identifies it. It typically gets assigned on creation kind of like a database's autonumber feature. We use this later so we can refer back to the associated worker message when the worker sends back something like progress for example. That way, we can tie the progress to a particular message by its `Id`.

There are two forms of message in this implementation - the one for workers and the one for the client. The one for the workers is significantly more verbose as it (often) contains the worker id and always contains the message id, whereas neither are tracked for the client messages since it doesn't need them. Each type of message takes one argument, and a command id that tells it what to do, although the client message just uses a `KeyValuePair<int,object>` to represent it. The worker message uses the `WorkerMessage` class to represent it.

It should be noted that in some cases, the client posts a message to itself. This is so whatever method that posted it can remain thread safe. Therefore, the method `DispatchWorkerMessage()` for example, is thread safe. If we had not posted the message to ourselves, then the dispatching would not be thread safe.

Thread Pooling

In order to utilize the CPU efficiently, we allocate a configurable number worker thread that defaults to the number of total cores minus one** clamped to a minimum of one. Each worker thread spins a loop until it receives a stop message, which keeps it alive until we tell it to shut down. We start a new worker thread any time there's no available worker threads to handle a request. This discounts threads that are currently busy, so if everything is busy, it will create a new worker thread if it can. Whether it can or not depends on whether we've met our quota for thread usage. If we can't allocate a new worker thread, we instead simply queue the message for pickup by the next worker thread that becomes available. In order to facilitate this, we track the number of idle worker threads and the total number of threads.

Technically, due to the limitations of a Windows Form app (it already spins its own loop out of the UI thread), you can't use the UI thread on the same thread as the client thread, so we really should be subtracting an extra core and therefore worker from the default pool size, but in the demo it doesn't bother. In a console app or a windows service, you can spin the client loop on the main thread, either in `Main()` or in `OnStart()` respectively and do the client logic from inside there. This is how I recommend doing it, when possible. There's a way to use the WinForms UI thread but it's fraught with limitations and complications. If you really want to do it, you can adapt the code from this article. You'd need the workers to communicate with the UI thread the client runs on using window messages instead of one of our message queues from above.

**The reason we allocate the number of workers as the number of cores minus one is because we don't want to stress the scheduler out, and our client has its own thread not counted among the workers.

If you're paying close attention, you'll note that where the rubber meets the road, the worker threads can continue to have queued messages even after no more workers can be created. They will be executed as I said, as soon as one of the workers becomes available. The more you increase the maximum messages, the more parallel your app will be, but after the default maximum you won't gain any performance - or at least you shouldn't.

Synchronization of Primitives

Aside from the message passing, we also use `Interlocked` to safely modify members and static members both of which might be accessed from different threads. We use this primarily for statistics, like the number of pending messages, or the number of available workers. This is just so those properties are thread safe.

The User Interface

The user interface goes about what it does a couple of ways - it polls for statistics from our client using a timer, and it also responds to a progress event that is fired from the client. This is so it can keep the UI up to date. The progress for each task is reported below the global statistics. It uses the progress event to tie the message `Id` back to the associated progress bar. That id is returned from the client whenever we dispatch a message. Whenever we get a progress event, we check that id and use a dictionary that maps the message `Id`s to `WorkerProgressControl`s which contain our progress bar. Note that the `WorkerMessageProgress` event is not fired on the UI thread but on the client thread instead. Therefore, we basically use another synchronization mechanism that `Control` has built in in order to handle the event.

Coding this Mess

The Worker

The `Worker` is the simpler of the two things and will introduce the message processing concept, so we'll start there.

The worker's job is to dispatch messages that come in on the queue and do different things depending on what command it was given. We accomplish this by spinning a `while` loop, wherein we `Wait()` on the `Client`'s `_messagesAvailable` class member (a `SemaphoreSlim` object) before finally switching on the message's `CommandId` to decide what to do.

This all happens in that `Start()` method, which blocks while spinning the above loop. We call start from a new thread whenever we create a new worker, like so:

C#

```
new Thread(() => { worker.Start(); }).Start();
```

Meanwhile, here's `Start()`:

C# Shrink ▲

```
// spin the main loop to keep the
// thread alive and processing
// messages.
var done = false;
while(!done)
{
    // wait until a message becomes available
    _messagesAvailable.Wait();
    WorkerMessage smsg;
    // check again just to be sure there's still
    // something there
    if(!done && _messages.TryDequeue(out smsg))
    {
        // tell the client we received a message
        _client.PostMessage(new ClientMsg(CLIMSG_MESSAGE_RECEIVED,
                new WorkerMessage(smsg.Id, Id, smsg.CommandId, smsg.Argument)));
        // TODO: Add your own messages here
        // you may want to make consts for
        // them
        switch(smsg.CommandId)
        {
            // TODO: replace below with
            // actual work
            case MSG_WORK: // do work
                // signal start of work
                _client.PostMessage(new ClientMsg(CLIMSG_PROGRESS,
                        new KeyValuePair<WorkerMessage, float>(smsg, 0f)));
                // simulate work:
                for (var i = 0;i<50;++i)
                {
                    Thread.Sleep(100);
                    // report some progress
                    _client.PostMessage(new ClientMsg(CLIMSG_PROGRESS,
                            new KeyValuePair<WorkerMessage,float>(smsg,(i+1)/50f)));
                }

                break;
            case MSG_STOP: // shut down
                // signalling shut down is simple
                done = true;
                break;
        }
        // tell the client we processed a message
        _client.PostMessage(new ClientMsg(CLIMSG_MESSAGE_COMPLETE,
                new WorkerMessage(smsg.Id,Id,smsg.CommandId,smsg.Argument)));
    }
}
```

`Client.PostMessage()` adds a message to the client's queue, and increments the client's message semaphore, signalling message(s) available. We use this to facilitate sourcing events like `WorkerMessageProgress` from the task client. Both `Client` and `Worker` have a `PostMessage()` method that is asynchronous and thread safe. They are critical to the message queue operation.

There's a way to do this which doesn't require the client to have a message queue but each worker would have to update the UI itself. There's an advantage to that approach which is that it avoids a potential bottleneck in the client wherein all of a sudden, it has to take time away from whatever else it's doing to process messages. The advantage to the approach we took however, is it does bidirectional communication which I wanted to illustrate here, but it does require the client to spin a message loop similar to the above. We'll explore it soon.

The message loop is implemented such that it forces the client thread to only "wake up" when it receives a message. In this way, it behaves like a `worker` does where it sits idle until a message is received. This may not be what you need. We'll discuss changing that behavior later.

Either way, the idea here is to replace that `for` loop and everything inside it with your own long running work, ideally passing progress back periodically as above.

Note that when we post the `CLIMSG_MESSAGE_COMPLETE`, it causes the client to raise the `WorkerMessageComplete` event. Also note that we are actually creating a new `WorkerMessage` out of the old one, and we're passing several items into the constructor. This is because when you use `DispatchWorkerMessage()`, the `WorkerMessage` you pass it will not have the worker's `Id` associated with it since you don't have one until a message gets handled by a worker. Since the `struct` is read only, we recreate it with the worker's `Id`.

The only other significant part of the worker is the initialization. It takes a reference to the client, and the two components of the workers' message queue:

C#

```
public Worker(Client client,SemaphoreSlim messagesAvailable,
              ConcurrentQueue<WorkerMessage> messages)
{
    _id = _NextWorkerId;
    Interlocked.CompareExchange(ref _NextWorkerId, 0, int.MaxValue);
    Interlocked.Increment(ref _NextWorkerId);
    _client = client;
    _messagesAvailable = messagesAvailable;
    _messages = messages;
}
```

The deal with `_NextWorkerId` is that it tracks possible new ids for each message that is created. It is a `static` member that gets incremented in a thread safe manner each time a `Worker` is created. It skips zero because we don't want zero `Id`s, just because we don't.

The Client

The client has a lot more responsibilities than the worker and is consequently quite a bit more complicated. It has to dispatch messages, manage the thread pool and track statistics like available workers. Of these, the thread pooling is the most complicated. Let's get it out of the way by exploring the `Start()` which behaves similarly to the worker's method of the same name. It blocks and waits on messages until it gets a `Stop()` message. Let's take a look:

C# Shrink ▲

```
// spin the loop
var done = false;
while (!done)
{
    // wait for an incoming message
    _messagesAvailable.Wait();
    ClientMsg climsg;
    if (_messages.TryDequeue(out climsg))
    {
        // TODO: Add your own messages here
        // you may want to make consts for
        // them
        switch (climsg.Key)
        {
            // a worker has received a message and has
            // just started processing it. The single
            // parameter (not used) is the message
            case MSG_MESSAGE_RECEIVED:
                // increment the pending message count
                Interlocked.Increment(ref _pendingWorkerMessageCount);
                // now a worker is busy so decrease our available
                // worker count
                Interlocked.Decrement(ref _availableWorkerCount);
                break;
                // a worker has completed processing a message
            case MSG_MESSAGE_COMPLETE:
                // decrease the pending worker count
                Interlocked.Decrement(ref _pendingWorkerMessageCount);
                var wrkmsg = (WorkerMessage)climsg.Value;
                if (WMSG_STOP == wrkmsg.CommandId)
                {
                    // if the worker sent sent a stop message
                    // decrement the worker count
                    Interlocked.Decrement(ref _workerCount);
                }
                else // otherwise increment the available workers
                    Interlocked.Increment(ref _availableWorkerCount);
                // raise the completed event
                WorkerMessageComplete?.Invoke
                      (this, new WorkerMessageCompleteEventArgs(wrkmsg));
                break;
                // a worker has progress to report
            case MSG_PROGRESS:
                var arg = (KeyValuePair<WorkerMessage, float>)climsg.Value;
                WorkerMessageProgress?.Invoke
                      (this, new WorkerProgressEventArgs(arg.Key, arg.Value));
                break;
```

```
            case MSG_DISPATCH: // dispatch a message to a worker
                // this message is invoked by DispatchWorkerMessage()
                // we handle it here to keep that message thread safe

                // if there are not available workers...
                if (0 == _availableWorkerCount)
                {
                    // and we haven't met our thread quota...
                    if (_workerCount < _maxWorkerCount)
                    {
                        // create a new worker
                        Interlocked.Increment(ref _workerCount);
                        Interlocked.Increment(ref _availableWorkerCount);
                        var ts = new Worker(this, _workerMessagesAvailable,
_workerMessages);
                        // start it
                        new Thread(() => { ts.Start(); }).Start();
                        // send it the message we received from the
                        // DispatchWorkerMessage()
                        _PostWorkerMessage((WorkerMessage)climsg.Value);
                    }
                    else
                    {
                        // we have to queue to an
                        // already busy worker.
                        _PostWorkerMessage((WorkerMessage)climsg.Value);
                    }
                }
                else
                {
                    // there's a worker available
                    // just post to it
                    _PostWorkerMessage((WorkerMessage)climsg.Value);
                }

                break;
                // called in response to Stop()
            case MSG_STOP:
                // we need to make sure we send as many stop messages as there are workers
                for (var i = 0; i < _workerCount; ++i)
                {
                    // increment the pending message count
                    Interlocked.Increment(ref _pendingWorkerMessageCount);
                    // post stop to the workers
                    _PostWorkerMessage(new WorkerMessage(WMSG_STOP, null));
                }

                // TODO: Wait until all work stops
                done = true;
                break;
        }
    }
}
```

Hopefully, the comments make it clear what it's doing. `_PostWorkerMessage()` is similar to `PostMessage()` in that it adds a message to the queue and increments the associated semaphore's

count. The only difference is `PostMessage()` works on the client's queue, and `_PostWorkerMessage()` works on the workers' queue. A lot of what we're doing is bookkeeping for the statistics. Notice we use `Interlocked` a lot. That is so we can safely set the values even if they're being accessed from another thread.

Note that when we handle `MSG_STOP`, we forward a `WMSG_STOP` message for each worker. This is so they all exit, and gracefully.

Remember what I said about changing the thread's behavior so it doesn't fall asleep until it receives a message? Let's say you're already spinning a tight loop on a thread, doing some processing, and the thread can't afford to wait for messages. Simply remove the semaphore associated with the queue, and remove the `Wait()` call and `Release()` call associated with it. Just set the loop to run without the semaphore. This won't work as well if your thread is waiting for other things too, as during that wait, it can't process more messages.

The User Interface

In the user interface, we provide facilities to queue a message (work item) to perform some faux "work", to examine the overall statistics, set the maximum threads in the pool, and to see progress for each queued work item.

We use a `WorkerProgressControl UserControl` to create a simple progress bar with a label to the left of it. The progress bar sizes with the control. The control itself has a custom constructor that takes a message id (the UI calles this a task id) which it then displays to the left of it. It should be noted that the progress bar is invisible until the first time the progress `Value` is set. Behind it, there's a label that tells the user it's queued and waiting. This is because it starts up queued and waiting and we want it to inform the user of that.

On the `Main Form`, we use a timer to poll for overall statistics in lieu of events. Things are actually simpler that way, and it works fine. We also have a control to edit the number of workers we can have. Finally, we have a `Panel` at the bottom that sizes with the form and auto-scrolls. It gets populated with new `WorkerProgressControl`s that get docked in the panel as each as item gets enqueued. A dictionary that maps message ids to progress controls is added to whenever an item gets enqueued.

When the `_client` raises the `WorkerMessageProgress` event, we handle the event on the main thread using `Control.BeginInvoke()` so that we can interact with the UI on its own thread for safety. We use the dictionary from earlier to match the incoming `Id` to a control and then we update that associated control's `Value`.

One weird thing we have to deal with is the case where we decrease the maximum worker count. In order to facilitate this, we must stop workers. For example, if we went from five workers to three, we'd need to stop two workers at the next opportunity (when two become idle). This is what `Client.DeallocateWorkers()` does.

First in the user interface, we have the main form's constructor code:

C# Shrink ▲

```
// designer support:
InitializeComponent();
// we allow the format string
// to be set in the designer
// this stores it for later
// because we change the text
// of the labels
_currentWorkersFmt = CurrentWorkersLabel.Text;
_waitingWorkItemsFmt = WaitingWorkItemsLabel.Text;
_pendingWorkItemsFmt = PendingWorkItemsLabel.Text;
_availableWorkersFmt = AvailableWorkersLabel.Text;

// hook the client's progress reporting
_client.WorkerMessageProgress += _client_WorkerMessageProgress;

// set the max worker box to the default max worker count
MaximumWorkersUpDown.Value = _client.MaximumWorkerCount;
// this is so we know when we've decreased the max workers
_oldMaximumWorkerCount = _client.MaximumWorkerCount;

// start the client thread so it will process messages
new Thread(() => { _client.Start(); }).Start();

// start the timer to keep the UI fresh
StatusTimer.Enabled = true;
```

The comments should make things clear. Remember we have to spin up a message loop in a thread in order to process worker messages. We must do similar with the client. That's what `Start()` does above.

Any time we get a progress message, we must `BeginInvoke()` to safely access the controls on the main UI thread since we're on a different thread inside this event. All we do is check the dictionary and then update the associated progress bar:

C#

```
BeginInvoke(new Action(delegate () {
    WorkerProgressControl wpc;
    if(_progressMap.TryGetValue(args.Id,out wpc))
    {
        wpc.Value = args.Progress;
    }
}));
```

Updating the UI with the timer even is trivial - no synchronization is required since the events fire on the main UI thread - here, we use the format strings we got from the designer on form start up:

C#

```
CurrentWorkersLabel.Text = string.Format(_currentWorkersFmt, _client.WorkerCount);
WaitingWorkItemsLabel.Text = _
```

```
    string.Format(_waitingWorkItemsFmt, _client.WaitingWorkerMessageCount);
PendingWorkItemsLabel.Text = _
    string.Format(_pendingWorkItemsFmt, _client.PendingWorkerMessageCount);
AvailableWorkersLabel.Text = _
    string.Format(_availableWorkersFmt, _client.AvailableWorkerCount);
```

Finally, when we click the **Enqueue** button, we must dispatch a work message to the workers, and then add a new `WorkerProgressControl` to our `Panel` from earlier. Note that `DispatchWorkMessage()` returns the `Id` of the newly created message:

C#

```
var id = _client.DispatchWorkerMessage(new WorkerMessage(MSG_WORK,null));
var wpc = new WorkerProgressControl(id);
// add the id to control mapping:
_progressMap.Add(id, wpc);
ProgressPanel.SuspendLayout();
ProgressPanel.Controls.Add(wpc);
wpc.Dock = DockStyle.Top;
ProgressPanel.ResumeLayout(true);
```

One way to improve this would be prioritization of task, but that adds significant complexity to the project.

History

- 16th July, 2020 - Initial submission

License

This article, along with any associated source code and files, is licensed under The MIT License

Written By

honey the codewitch

United States

Just a shiny lil monster. Casts spells in C++. Mostly harmless.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

Nice...
Michael Breeden 18-Jul-20 0:38

Re: Nice...
honey the codewitch 18-Jul-20 2:28

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2020 by honey the codewitch
Everything else Copyright © CodeProject, 1999-2022

Web04 2.8:2022-10-01:1